Operational semantics for Prolog with Cut in Rocq and its application to determinacy analysis

Anonymous author
Anonymous affiliation
Anonymous author
Anonymous affiliation

Abstract

We mechanize two operational semantics for Prolog with cut and prove them equivalent in the Rocq prover. The first semantics closely models Elpi's implementation: it is based on a stack of alternatives and is well suited for studying optimizations employed by Prolog abstract machines. The second semantics is instead based on And-Or trees, in which the branches pruned by the cut operator are made explicit.

We use the tree-based semantics to (i) describe introduction and elimination rules for disjunction in the presence of cut, which (as expected) depart from those of classical logic, and (ii) reason about the determinacy analysis introduced in Elpi version 3.0, which statically identifies computations that produce at most one result.

2012 ACM Subject Classification Theory of computation → Operational semantics

Keywords and phrases Semantics, Prolog, Elpi, Determinacy Analysis, Rocq, Coq

Digital Object Identifier 10.4230/LIPIcs.CVIT.2016.23

1 Introduction

Elpi is a dialect of λprolog (see [19, 20, 9, 14]) used as an extension language for the Rocq prover (formerly the Coq proof assistant). Elpi has become an important infrastructure component: several projects and libraries depend on it [15, 3, 4, 26, 10, 11]. Other remarkable libraries include the Hierarchy-Builder library-structuring tool [5] and Derive [24, 25, 13], a program-and-proof synthesis framework with industrial applications at SkyLabs AI.

Starting with version 3, Elpi is equipped with a static analysis for determinacy [12] to help users control backtracking. Rocq users are familiar with functional programming, but not necessarily with logic programming; in particular, uncontrolled backtracking is a common source of inefficiency and makes debugging harder. The determinacy checker allows the user to assert which predicates should be considered deterministic and then checks that these predicates behave like functions, i.e., they commit to their first solution and leave no *choice points* (places where backtracking could resume). A choice point correspond to an suspended *alternative* in the execution trace. In the following we use *choice point* and *alternative* as synonyms.

This paper reports our first steps towards a mechanization, in the Rocq prover, of the determinacy analysis from [12]. We focus on the control operator *cut*, which is useful to restrict backtracking but makes the semantics depart from a pure logical reading.

We formalize two operational semantics for Prolog with cut. The first is a stack-based semantics that closely models Elpi's implementation and is similar to the semantics mechanized by Pusch in Isabelle/HOL [21] and to the model of Debray and Mishra [6, Sec. 4.3]. This stack-based semantics is a good starting point to study further optimizations used by standard Prolog abstract machines [27, 1], but it makes reasoning about the scope of cut difficult. To address this limitation we introduce a tree-based semantics in which the branches pruned by cut are explicit and we prove the two semantics equivalent. Using the

```
func f int -> int.          % f is a deterministic predicate. The two rules
f 1 2.                      % have non-overlapping heads, i.e. 1 != 2
f 2 3.
pred r int -> int.          % r is a non-deterministic predicate. The heads
r 0 0.                      % overlap: the query for 0 has three applicable
r 0 1.                      % rules.
r 0 2 :- !.
func g int -> int.          % g is deterministic: if the first rule succeeds, the
g A C :- r A B, f B C, !.   % second one is cut away, as well as the choice
g D F :- f D E, f E F.      % points left by the call to r.
```

Figure 1 Running example of a ELPI program

tree-based semantics, we then prove some properties about the impact of evaluating the cut in disjunctive querys. For example, unlike in classical logic, $q_1 \vee q_2$ is not equivalent to $q_2 \vee q_1$, since q_2 may be cut-away by q_1.

Finally, we use the formal semantics to prove the correctness of a static determinacy analysis [12]. Intuitively, nondeterminism arises when a computation can be completed by applying two different rules. We exclude this situation in two ways. Firstly, at most one rule should be used on a given query. This condition is satisfied if (i) the rules have non-overlapping heads: if two heads do not overlap in the inputs they accept, then no query can unify with both, or if (ii) we account for the presence of cut in rule premises: once a rule whose premises contain a cut succeeds, all remaining rules are discarded. Secondly, each rule of a deterministic predicate should (i) either call functions, this ensure that no choice point is left after the execution of the premises of the chosen rule, (ii) or call relations provided that they are followed by the cut operator, so that any allocated choice points preceeding the cut are discarded.

We show that if every rule defining a deterministic predicate passes this analysis, then any call to that predicate leaves no choice points.

2 Preliminaries: syntax and informal semantics of cut

In the entire paper we use Figure 1 as our running example, and we start by describing how we represent a program like that one. In the concrete syntax, variables are written with capital letters, and predicate application follows the λ-calculus style (no parentheses).

The description of a program is given by the record $\mathbb{P}$ and is shared by both semantics. A program consists of a list of rules `R` together with a signature environment `sigT`. We delay the descussion of signatures to Section 6 where we describe the static analysis that concerns them.

A rule consists of a head term `Tm` and a list of premises. Each premise is an `Atom`, i.e. either the cut operator or a predicate call. Terms `Tm` include predicate symbols, data symbols,

```
Inductive Tm := Symb of S | Pred of P | Var of V | App of Tm & Tm.
Inductive Atom := cut | call of Tm.
Record R := mkR { head : Tm; premises : list Atom }.
Record ℙ := { rules : seq R; sig : sigT }.
Definition Σ := {fmap V -> Tm}.
```

Figure 2 Definition of term, rule, program and substitution

and variables. The syntax tree allows variables to be applied and predicates to be mixed with data. These higher order features play no role in the current paper that focuses on first order logic programming, but allows future extensions to full λProlog to be based on the same mechanization.

The set of variables `V`, predicates `P` and data symbols `S` are countable. We represent substitutions Σ as finitely supported maps from variables to terms, using the finmap library [18]. The empty substitution is written ε.

The backchaining operation applies all rules to a query term; it is central to both semantics.

```
Definition backchain : ℙ → 𝓕ᵥ → Tm → Σ → 𝓕ᵥ * list (Σ * list Atom)
```

Since a rule may be applied multiple times, its variables must be freshened before each application. Accordingly, the backchain function takes a program, the set of variable names used so far ($\mathcal{F}_\nu$), a query term, and the current substitution. It returns an updated set of variable names together with the list of alternatives, i.e. the rules that apply. More precisely, for each applicable rule it returns the rule premises together with an updated substitution obtained by unifying the rule head with the query term.

In our mechanization, we abstract over the unifier and its properties. The function `unify` takes two terms t_1 and t_2, together with a substitution σ, as input, and optionally returns a substitution σ' that extends σ. The expected behavior of `unify` is the standard one (see for example [8, section 2.5]), limited to the first order. Therefore, if t_1 and t_2 unify under σ, then $\sigma'\ t_1 = \sigma'\ t_2$, where $\sigma\ t$ denotes subsitution application and $=$ denotes syntactic equality.

2.1 The cut operator

There are quite a few variants of the cut operator. The one we are interested in, i.e. the one used in Elpi, is known as *hard cut* (see, e.g., the documentation of SWI-Prolog [23]). Whenever the `backchain` function returns a non-empty list of alternatives, the first one is explored immediately, while the others are suspended as choice points. Within a given alternative, premises are processed from left to right, executing each atom in turn. When a cut is encountered, it discards (i) all choice points created by `backchain` for the current call and (ii) all choice points created while executing the premises to the left of the cut.

For example, consider the program in Figure 1 and the query `g 0 R`. Both rules for `g` apply to this query. Backchaining therefore returns the following two alternatives:

$$[(\sigma_1, [\texttt{r A B, f B C, !}]),\ (\sigma_2, [\texttt{f D E, f E F}])]$$

with $\sigma_1 \coloneqq \{\texttt{A} \mapsto \texttt{0}, \texttt{C} \mapsto \texttt{R}\}$ and $\sigma_2 \coloneqq \{\texttt{D} \mapsto \texttt{0}, \texttt{F} \mapsto \texttt{R}\}$. For simplicity, we choose an example where variable refreshing plays no role, so we do not discuss the set $\mathcal{F}_\nu$ any further.

Execution continues with the first premise of the first alternative, namely `r A B` under σ_1, i.e. `r 0 B`. The remaining alternatives are stored as choice points. Backchaining `r 0 B` returns $[(\sigma_3, []), (\sigma_4, []), (\sigma_5, [\texttt{!}])]$ where $\sigma_3 \coloneqq \sigma_1 + \{\texttt{B} \mapsto \texttt{0}\}$; $\sigma_4 \coloneqq \sigma_1 + \{\texttt{B} \mapsto \texttt{1}\}$ and $\sigma_5 \coloneqq \sigma_1 + \{\texttt{B} \mapsto \texttt{2}\}$.

The next step of interpretation continues with `f B C` under σ_3, that is, `f 0 R`, and leaves $[(\sigma_4, []), (\sigma_5, [\texttt{!}])]$ as new choice points. This time, backchaining fails (no rule for `f` matches input 0). We therefore backtrack to the most recently introduced choice point and retry `f B C` under σ_4, i.e. `f 1 R`. This succeeds with $\sigma_6 \coloneqq \sigma_4 + \{\texttt{R} \mapsto \texttt{2}\}$.

We now reach the cut. At this point there are still two unexplored alternatives: one from the third rule for `r` and one from the second rule for `g` (respectively, σ_5 and σ_2). Both are

discarded, because they were created when, or after, the first rule for `g` was selected. In contrast, a cut does not discard choice points created earlier; in this example, there are none. The final solution is: $\{\texttt{A} \mapsto \texttt{0}, \texttt{B} \mapsto \texttt{1}, \texttt{C} \mapsto \texttt{R}, \texttt{R} \mapsto \texttt{2}\}$.

In Sections 3 and 4, we provide fully mechanized operational semantics for Prolog with cut. They differ in how they represent the ongoing computation, in particular how alternatives are stored and how they are pruned by cut.

Notational conventions In the following section we note $\mathbb{B}$ for the bolean type, and $\top$ and $\bot$ for `true` and `false`. The constructors of the option type are written $\cdot t$ for `Some` t and $\square$ for `None`. Following the Mathematical Components library, on which we depend, we use `x.1` and `x.2` to denote the first and second projection applied to the pair `x`. List concatenation is the infix `++`, while list comprehension (i.e. maping a function on a list) is written `[seq f x | x <- l]`.

3 And-Or Tree semantics

An And-Or tree is a stratified, variable-width tree, defined in Rocq as in Section 3. It consists of two kinds of layers that alternate: a disjunctive layer is followed by a conjunctive layer and vice versa. At the root (a query), the tree records a list of alternatives (an Or-layer); for each alternative, it records a list of subqueries to be solved (an And-layer). Intuitively, solving a query requires solving one alternative in the Or-layer, but all subqueries in the corresponding And-layer.

The tree is built incrementally. The `Todo` node represents an unexplored branch (an `Atom`). When the atom is a `call`, we use the backchaining procedure from Section 2 to compute the two layers to attach to the tree: alternatives form the Or-layer, and the premises of each applicable rule form the corresponding And-layer. This expansion step is performed by the `step` procedure described in Section 3.2.

In addition to the `Todo` node we have two distinguished nodes, `KO` and `OK`, which represent respectively the smallest failed and successful tree.

The `Or` node, written $A \vee B_\sigma$, represents the addition of an alternative B_σ to an existing disjunction A. The left subtree A is optional and is absent once that branch has been fully explored. The right alternative is paired with a substitution σ, which becomes the current substitution when backtracking to B.

The `And` node, written $A \wedge_{B_0} B$, represents the addition of a subquery B to the first choice point in A. We call B_0 the *reset point* for B: it represents the continuation for all alternatives of A except the first. That is, when backtracking occurs within A, the subtree B is replaced by the atoms in B_0. An example is shown in Section 3.3.

A graphical representation of a tree is shown in Figure 5b. For compactness, we depict `And` and `Or` nodes as n-ary, with children associating to the right. The `KO` node acts as the terminating element of an `Or` list, while `OK` is the terminating element of an `And` list.

```
Inductive tree :=
  | KO | OK                          (* failure and success *)
  | Todo of Atom                     (* unexplored branch *)
  | Or   of option tree & Σ & tree   (* A ∨ B_σ *)
  | And  of tree & list Atom & tree. (* A ∧_B0 B *)
```

Figure 3 Definition of And-Or tree

3.1 The semantics

The tree is constructed incrementally by two recursive functions, `step` and `prune`. Both traverse the tree depth-first, left-to-right. The node to be explored in a tree is retrieved thanks to the `next_tree` (in Figure 4). When this node is `OK` we say the entire tree is a *success*, when it is `KO` we say the tree *failed*, otherwise the tree is *incomplete*. In Figure 5 we mark with a black arrow the result of `next_tree`.

```
Inductive tag := Expanded | CutBrothers | Failed | Success.
Definition step : ℙ → 𝓕ᵥ → Σ → tree → (𝓕ᵥ * tag * tree)
Definition prune : 𝔹 → tree → option tree
Inductive runT : ℙ →  𝓕ᵥ → Σ → tree → option (Σ * option tree) → 𝔹 → 𝓕ᵥ → Prop
```

Since all functions must terminate in Rocq, we define their transitive closure of `step` and `prune` as the inductive relation `runT`. More precisely `runT` iterates calls to `step` on incomplete trees to expand `Todo` nodes. When a tree fails it calls `prune` to find the next alternative and continues from there, if one exists.

3.2 The `step` procedure

The `step` procedure takes as input a program, a set of variables, a substitution, and a tree, and returns an updated set of variables, a `tag`, and an updated tree.

The set of variables is assumed to contain all variables occurring in the tree. It is passed to backchain so that rules can be refreshed correctly.

The `tag` indicates which kind of step was performed. `Failure` and `Success` are returned for trees that satisfy, respectively, the `failed` and `success` tests. `CutBrothers` indicates the interpretation of a superficial cut: `step` was called on an And-layer and its `next_tree` is the cut atom. Finally, `Expanded` accounts for the interpretation of predicate calls and non-superficial cuts.

The `step` procedure evaluates atoms. In particular, it leaves the tree unchanged when the tree is *success* or *failed*.

▶ **Lemma 1** (succ_step_iff). `∀ p v σ t, success t ↔ step p v σ t = (v, Success, t).`

▶ **Lemma 2** (fail_step_iff). `∀ p v σ t, failed t ↔ step p v σ t = (v, Failed, t).`

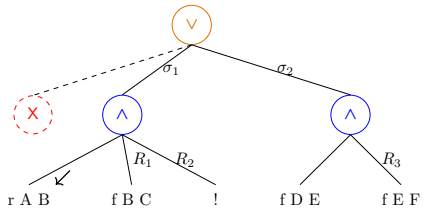
The `step` procedure expands `Todo` nodes by calling `backchain`, which returns a list l of alternatives. If l is empty, then the current call atom is replaced by `KO`. Otherwise, it is replaced by a right-skewed tree of `Or` nodes, where each leaf corresponds to a list of atoms $e \in l$. If e is empty, the leaf is `OK`; otherwise, it is a right-skewed tree of `And` nodes.

```
Fixpoint next σ t : Σ * tree :=
  match t with
  | Todo _ | KO | OK ⇒ (σ, t)
  | Or □ σ' B ⇒ next σ' B
  | Or ·A _ _ ⇒ next σ A
  | And A _ B ⇒
    let (σ', pA) := next σ A in
    if pA == OK then next σ' B
    else (σ', pA)
  end.
```

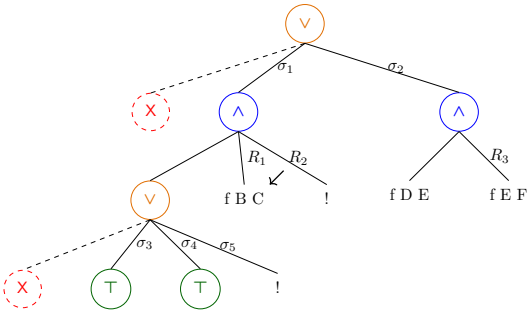
```
Definition next_subst σ t := (next σ t).1.
Definition next_tree t := (next ε t).2.

Definition success t := next_tree t == OK.
Definition failed t := next_tree t == KO.
Definition incomplete t :=
    if next_tree t is Todo _ then ⊤ else ⊥.
```

Figure 4 `next` and derived functions

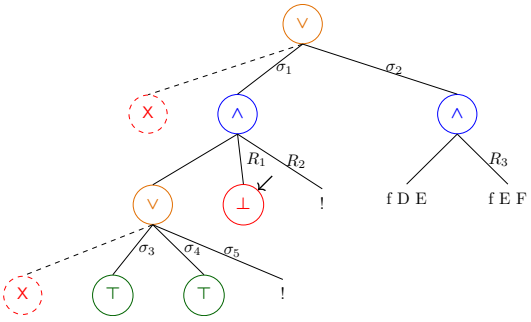
(a) Initial query



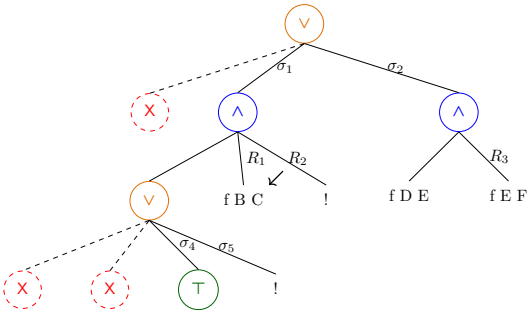
(b) Tree after `step` on `g 0 R`



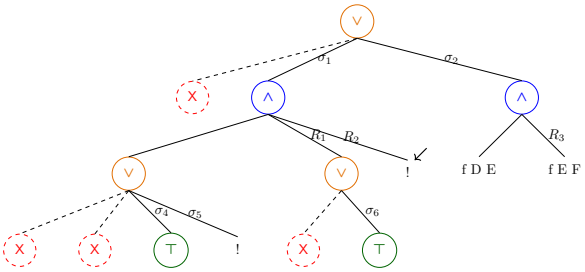
(c) `step` on Figure 5b



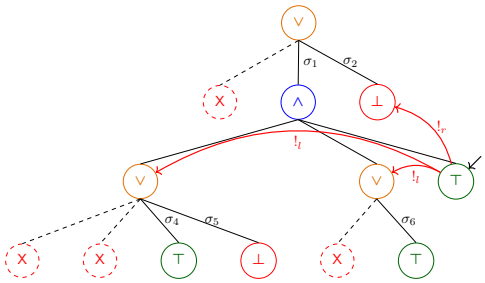
(d.1) `step` on Figure 5c



(d.2) `prune` on Figure 5d.1



(e) `step` on Figure 5d.2



(f) `step` on Figure 5e

Figure 5 Execution in the tree semantics. The substitutions $\sigma_1 \dots \sigma_6$ are from Section 2.1. R_1, R_2, and R_3 are reset points and correspond respectively to the lists of atoms [`f Y Z, !`], [`!`], and [`f Y Z`].

Figure 5 depicts the tree corresponding to the running example (Section 2.1) as it undergoes calls to `step`. We run query `q 0 R` against the program P in Figure 1. Let c_0, c_1, and c_2 be the children of the root. By construction, c_0 is the $\Box$ tree, while c_1 and c_2 correspond respectively to the premises of rules r_1 and r_2 in P. Note that c_1 (resp. c_2) is associated with substitution σ_1 (resp. σ_2), whose values are the same as in Section 2.1. Reset points are defines as follows: $R_1 \coloneqq$ [`f Y Z, !`], $R_2 \coloneqq$ [`!`], and $R_3 \coloneqq$ `f Y Z`. Intuitively, they record the lists of atoms used to generate the corresponding subtree. Other examples of `step` performing a tree expansion via backchain appear in Figures 5c–5e.

3.2.1 The interpretation of a cut

The step corresponding to a cut performs two actions: (i) the cut node is replaced by `OK`; and (ii) the subtrees in the scope of `Cut` are pruned. More precisely, (ii) prunes the left siblings of the `Cut` node (in the same And-layer, denoted $!_l$) and prunes the right uncles of `Cut` (in the one-level-up Or-layer, denoted $!_r$).

An example of the impact of cut is shown in Figure 5f. The red arrows indicate the scope of the cut and are labeled with $!_r$ and $!_l$ to indicate which pruning operation is applied to each pointed subtree. Note that the evaluation of a cut keeps the path leading to the cut node unchanged, in the sense that substitution σ_4 is preserved, while the path under substitution σ_5 (which also succeeds) is replaced by `KO`. This amounts to discarding the third rule of predicate `r`.

3.3 The `prune` procedure

When `backchain` returns an empty list, `step` produces a tree that *failed* and the computation must backtrack. The `prune` procedure is responsible for producing the new tree from which the computation can continue.

In Figure 5d.1 we encounter a failure because no rule applies to the atom `f B C` under substitution σ_3, which maps `B` to `0`. The `prune` procedure prunes the path leading to this failure and resets the current sub-query to its original shape. More precisely, it kills the `OK` node under σ_3 (since that branch has been fully explored and produced no solution), and then replaces the `KO` node with the information stored in the reset point R_1. This restores the call `f B C`, which can then be reconsidered under substitution σ_4.

Furthermore, `prune` serves another important purpose. Its behavior depends on the boolean flag it receives as input: `prune` $\bot$ t recursively removes all failures (if any) from t, whereas `prune` $\top$ t removes the first success in t. For example, the tree in Figure 5f contains a successful path that maps `R` to `2`. Calling `prune` on this tree returns $\Box$, since there are no alternatives in the tree after the success.

Some interesting properties of `prune` are shown below; they illustrate how `prune` complements `step` with respect to failed, successful, and incomplete trees.

▶ **Lemma 3** (incomplete_prune_id). $\forall\ b\ t,\ \mathit{incomplete}\ t \to \mathit{prune}\ b\ t = \cdot t.$

▶ **Lemma 4** (prune_Some). $\forall\ b\ t\ t',\ \mathit{prune}\ b\ t = \cdot t' \to \mathit{failed}\ t' = \bot.$

▶ **Lemma 5** (prune_None). $\forall\ t,\ \mathit{prune}\ \bot\ t = \Box \to \mathit{failed}\ t.$

3.4 The `runT` relation and its properties

The inductive relation `runT` relates four inputs to three outputs. The inputs are a program, a set of variables, an initial substitution σ, and a tree t. The outputs are an optional result r, a boolean b, and an updated set of variable names.

$$\frac{\texttt{success}\ t \qquad \texttt{next\_subst}\ \sigma\ t = \sigma' \qquad \texttt{prune}\ \top\ t = t'}{\texttt{runT}\ \mathrm{p}\ v\ \sigma\ t\ {\cdot}(\sigma', t')\ \bot\ v}\ \mathit{StopT}$$

$$\frac{\begin{array}{cc}\texttt{incomplete}\ t & b' = (\mathrm{tg} = \texttt{CutBrothers}) \vee b \\ \texttt{step}\ \mathrm{p}\ v\ \sigma\ t = (v', \mathrm{tg}, t') & \texttt{runT}\ \mathrm{p}\ v'\ \sigma\ t'\ r\ b\ v''\end{array}}{\texttt{runT}\ \mathrm{p}\ v\ \sigma\ t\ r\ b'\ v''}\ \mathit{StepT}$$

$$\frac{\texttt{failed}\ t \qquad \texttt{prune}\ \bot\ t = {\cdot}t' \qquad \texttt{runT}\ \mathrm{p}\ v\ \sigma\ t'\ r\ \mathrm{n}\ v'}{\texttt{runT}\ \mathrm{p}\ v\ \sigma\ t\ r\ \mathrm{n}\ v'}\ \mathit{BackT}$$

$$\frac{\texttt{prune}\ \bot\ t = \square}{\texttt{runT}\ \mathrm{p}\ v\ \sigma\ t\ \square\ \bot\ v}\ \mathit{FailT}$$

Figure 6 Rule system for `runT`

The main result r is $\square$ when the execution fails; otherwise when $r = {\cdot}(\sigma', t')$ the execution succeeded and produced the solution σ'; t' represents the remaining, not-yet-explored alternatives. The boolean b records whether a superficial cut was evaluated during execution.

The `runT` predicate has four cases, depicted as inference rules in Figure 6. (*StopT*) If `next_tree` t is a success, then the substitution σ' is extracted from t (starting from σ) and the input tree is cleaned of its successful path. (*StepT*) If `next_tree` t is an atom, then `step` is invoked to evaluate t, and `runT` is called recursively on the updated tree. (*BackT*) If `next_tree` t is a failure, then `prune` is called to clear the failed path; if the resulting cleaned tree exists, `runT` is called recursively on it. Finally, (*FailT*) accounts for trees with no solutions.

The set of rules defining `runT` is deterministic.

▶ **Lemma 6** (runT_det). $\forall\ p\ v_0\ \sigma\ t\ r\ r'\ b\ b'\ v\ v',\ \mathit{runT}\ p\ v_0\ \sigma\ t\ r\ b\ v \rightarrow$
$\mathit{runT}\ p\ v_0\ \sigma\ t\ r'\ b'\ v' \rightarrow r' = r \wedge v' = v \wedge b' = b.$

The proof is by induction on the first `runT` derivation and is immediate, because the rules are selected based on `next_tree` t. In particular, the hypotheses `success` t, `incomplete` t, and `failed` t are mutually exclusive. Moreover, by Lemma 5, the hypothesis of *FailT* implies that t *failed*; hence *FailT* is exclusive with *StopT* and *StepT*, and it is also exclusive with *BackT* since they impose different requirements on the output of `prune`.

The boolean output of `runT` allows us to state interesting properties about the `Or` node in the presence of cut. Here we report only a selection of those we proved.

▶ **Lemma 7** (runSST_or). $\forall\ p\ v\ v'\ \sigma\ \sigma'\ A\ A'\ \sigma_1\ B,\ \mathit{runT}\ p\ v\ \sigma\ A\ {\cdot}(\sigma',\ {\cdot}A')\ \top\ v' \rightarrow$
$\mathit{runT}\ p\ v\ \sigma\ ({\cdot}A \vee B_{\sigma_1})\ {\cdot}(\sigma',\ {\cdot}({\cdot}A' \vee \mathit{KO}_{\sigma_1}))\ \bot\ v'.$

▶ **Lemma 8** (runNT_or). $\forall\ p\ v\ v'\ \sigma\ A\ \sigma_1\ B,\ \mathit{runT}\ p\ v\ \sigma\ A\ \square\ \top\ v' \rightarrow$
$\mathit{runT}\ p\ v\ \sigma\ ({\cdot}A \vee B_{\sigma_1})\ \square\ \bot\ v'.$

▶ **Lemma 9** (runNF_or). $\forall\ p\ v_0\ v_1\ v_2\ \sigma\ A\ \sigma_1\ \sigma_2\ B\ B'\ b,$

```
runT p v0 σ A □ ⊥ v1 → runT p v1 σ1 B ·(σ2, B') b v2 →
  let sR := omap (fun x ⇒ □ ∨ x_σ1) B' in
  runT p v0 σ (·A ∨ B_σ1) ·(σ2, sR) ⊥ v2.
```

Lemmas 7–9 correspond to the introduction rules for disjunction. If A executes a cut, one *cannot* obtain $A \vee B$ from B: the execution of A, whether it ultimately succeeds or fails, makes the success of B irrelevant (the cut discards it). In the first lemma, B is forced to be `KO`; in the second one, the failure of A absorbs B. The last lemma is connected to the depth-first exploration of the

tree: proving a disjunction from its right-hand side can only occur after the left-hand side has been disproved. Depicted as rules:

$$\frac{A \quad (A \text{ cuts})}{A \vee \top}\ (\vee_{il}) \qquad \frac{\neg A \quad (A \text{ cuts})}{\neg(A \vee B)}\ (\vee_{ir1}) \qquad \frac{\neg A \qquad B \quad (A \text{ does not cut})}{A \vee B}\ (\vee_{ir2})$$

Lemmas 10 and 11 are elimination rules for disjunction.

▶ **Lemma 10** (run_orSST). `∀ p v v' σ σ' σ₁ A A' B B',`

```
runT p v σ (·A ∨ B_σ₁) ·(σ', ·(·A' ∨ B'_σ₁)) ⊥ v' →
  ∃ b, runT p v σ A ·(σ', ·A') b v' ∧ B' = if b then KO else B.
```

▶ **Lemma 11** (run_orSNT). `∀ p v v' σ σ' σ₁ A B B',`

```
runT p v σ (·A ∨ B_σ₁) ·(σ', ·(□ ∨ B'_σ₁)) ⊥ v' →
  (∃ b, runT p v σ A ·(σ', □) b v' ∧ if b then B' = KO else prune ⊥ B = ·B') ∨
  (∃ v₂ b, runT p v σ A □ ⊥ v₂ ∧ runT p v₂ σ₁ B ·(σ', ·B') b v').
```

It is worth recalling that the "A cuts" side-condition in the rules above is precisely the boolean result returned by `runT`. Without this information, it is impossible to formulate these rules: it is not merely a matter of whether A contains a syntactic occurrence of cut, but whether that cut is actually executed during the (dis)proof of A (in rules $(\vee_{ir1})$ and $(\vee_{ir2})$). Indeed, an atom preceding the cut may fail, so the cut is never reached.

As anticipated in the introductionction these lemmas contradict the commutativity of disjunction.

4 Stack-based semantics

The stack-based semantics we present is very close to what is implemented by the Elpi interpreter. This semantics is similar to the one proposed by Pusch in [21], which is in turn closely related to the semantics given by Debray and Mishra in [6, Section 4.3]. Pusch mechanized this semantics in Isabelle/HOL, together with several optimizations that are also present in the Warren Abstract Machine [27, 1].

The execution state is a stack of alternarives. Each alternative is a list of goals. Intuitively it amounts to a disjunctive normal form: each stack item is a self contained computation, coupling a substitution with all the goals to be solved.

Goals pair an atom with a list of the *cut-to* alternatives, making the two datatypes mutually recursive, but these alternatives have a very different role. They have to be understood as pointers to lower levels of the stack itself. This datum is used to implement the cut, i.e. to prune the stack of alternatives.

```
Inductive alts  := nilA | consA of (Σ * goals) & alts
with       goals := nilG | consG of (Atom * alts)  & goals
```

The stack-based semantics is described by the `runS` inductive predicates.

```
Inductive runS (p: ℙ) : F_ν → alts → option (Σ * alts) → Prop
```

```
Definition save_gs a g b :=
  [seq (x, a) | x <- b] ++ g.

Definition save_as a g b :=
  [seq (x.1, save_gs a g x.2) | x <- b].
```

```
Definition stepS p v t σ a g :=
  let (v', r) := backchain p v t σ in
  (v', save_as a g r).
```

$$\frac{}{\texttt{runS p } v\ ((\sigma, \varnothing) :: a) \cdot(\sigma, a)}\ StopS \qquad \frac{\texttt{runS p } v\ ((\sigma, g) :: ca)\ \texttt{r}}{\texttt{runS p } v\ ((\sigma, (\text{cut}, ca) :: g) :: \_)\ \texttt{r}}\ CutS$$

$$\frac{\texttt{stepS p } v\ t\ \sigma\ a\ g = (v_1, a') \qquad \texttt{runS p } v_1\ (a' +\!\!+ a)\ \texttt{r}}{\texttt{runS p } v\ ((\sigma, (\text{call } t, \_) :: g) :: a)\ \texttt{r}}\ CallS \qquad \frac{}{\texttt{runS p } \_\ \varnothing\ \square}\ FailS$$

Figure 7 Rule system for `runS`

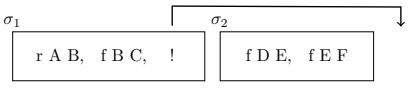
The `runS` predicates relates a set of variables and a list of alternatives to optional result. □ stands for failure, i.e., no solution, whereas $\cdot(\sigma, a)$ represent success with σ begin the solution and a as the list of non-yet-explored alternaitves.

The rule system for `runS` is given in Figure 7 and is made by 4 cases. We distinguish two main cases. If we run out of alternatives we fail: (*FailS*) stops the execution and return □. If the list of alternatives is $(\sigma_0, h) :: a$, then we look at the list of goals h (under the substitution σ_0). If h is empty, (*StopS*) stops the execution with a success σ_0 leaving a as the unexplored alternatives. If h is not empty we look at its head goal (t, ca) where t is the atom and ca is its cut-to alternatives. If t is a cut rule (*CutS*) continues to evaluate the tail of h under with alternatives ca. Finally, when t is a call, rule (*CallS*) backchains: for each rule it pairs each premise with the current stack of alternatives, and prepents the obtained goals to the tail of h. Note that if backain returns the empty list, (*CallS*) second premise amounts to exploring the next item in the current stack.

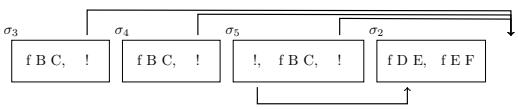
4.1 Example of execution

We propose an example of the execution of `runS` in Figure 8. The arrows in the images represent the cut-to pointers. We have not drawn arrows coming out of call atoms since the interpreter ignores the cut-alternatives in this case. The evolution of the stack in Figure 8 mirrors the example in Section 2.1: we evaluate the first goal of the first alternative. During backchaining, we add a pointer to the remaining alternatives to each new cut operator.

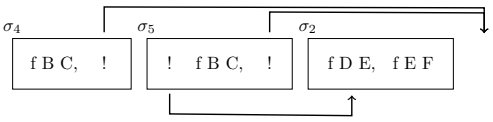
(a) Initial query



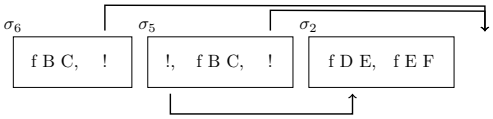
(b) Stack after call to `g 0 R`



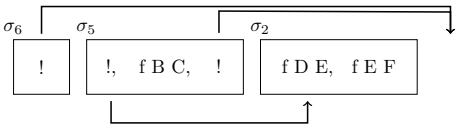
(c) Stack after evaluating Figure 8b



(d) Stack after evaluating Figure 8c



(e) Stack after evaluating Figure 8d



(f) Stack after evaluating Figure 8e

Figure 8 Example of execution

If a cut is evaluated, we replace the remaining alternatives with the cut-to alternatives. For example, in Figure 8f, evaluating the cut discards the second and third elements in the stack. Backtracking is performed by simply consuming the first alternative from the stack, as shown in Figure 8d.

4.2 Inadequacy of the stack-based semantics for formal reasoning

The lemmas that we presented in Section 3.4 are hard to express in the stack-based semantics, due to the lack of structure in the stack to delimit the scope of the cut operator. For example, consider a stack $a_0, a_1, \ldots, a_n$. If a_0 succeeds but executes a cut, it is hard to relate the remaining alternatives $a_1, \ldots, a_n$ to the alternatives $b_1, \ldots, b_m$ that survive the cut, since the only simple invariant is that $b_1, \ldots, b_m$ is a suffix of $a_1, \ldots, a_n$. If $a_1, \ldots, a_n$ were generated before the call whose execution generates the cut in a_0, then they all stay; but if some were generated afterwards, they are discarded. Expressing this precisely would require attaching, at the very least, time-stamps (or depths) to goals, which amounts to a cumbersome encoding of a tree.

5 Equivalence of the two semantics

In order to relate the two semantics we have to relate the computations states, i.e., the And-Or tree and the stack of alternatives. We define a translation from trees to stacks, called `t2l`, but we do not explcitly provide the converse translation, an economy allowed by the proof technique we employ, inspired by [2].

Before describing the translation of trees to stacks (in Section 5.2) we need to define the notion of *valid tree*. The `tree` datatype indeed contains trees that cannot be generated by `runT` via `step` or `prune`, for example all the trees that do not correspond to a depth-first left-to-right tree construction strategy.

5.1 Valid trees

The class of valid trees is captured by the `valid_tree` function shown in Figure 9. While all base cases of `tree` are valid, we need to analyze the `Or` and `And` constructors more carefully.

For the `Or` constructor, we distinguish two cases depending on whether the left-hand side is present. If it is not, then the right-hand side must be a valid tree. Otherwise, the left-hand side must be a valid tree, and the right-hand side must either be the `KO` tree or be unexplored. The former case occurs when the right-hand side is cut away by the evaluation of a superficial cut in the left-hand side. In the latter case we use the `base_or` predicate to check that `B` is the right-skewed tree formed by a disjunction of conjunctions, generated after a successful backchain for a `call`.

For the `And` constructor, the left hand side is required to be a valid tree. The shape of the right hand side depends on whether the left hand side is a success. If it is not successful, then the right hand side must not be explored, i.e. `B` must be the right-skewed tree containing conjunctions of premise atoms. This is enforced using the `big_and` procedure, which generates a tree from the reset point B_0 (the same procedure used to transform each alternative output by `backchain` into the And-layer of the resulting tree). When the left hand side is successful, then the right hand side may have been explored, and consequently must be a valid tree.

5.2 From trees to stacks

The `t2l` recursive function translates a tree to a stack. For space constraints Figure 9 only gives the base cases, while we describe the recursive cases in the following text.

The function `t2l` takes as input the tree t_0, a substitution σ_0, and a list of choice points cp. These choice points represent alternatives that appear at the same level of the current tree, such as, for example, the right siblings of an `Or` node. The substitution is used to determine under which substitution the subtree being computed lives.

The `OK` node represents one successful alternative; therefore it is translated into a singleton list whose only element has an empty list of goals. The `KO` node represents failure, hence it is

```
Fixpoint valid_tree A :=
  match A with
  | Todo _ | OK | KO ⇒ ⊤
  | Or □ _ B ⇒ valid_tree B
  | Or ·A _ B ⇒ valid_tree A &&
        ((B == KO) || B.base_or B)
  | And A BO B ⇒ valid_tree A &&
      if success A then valid_tree B
      else B == big_and BO
  end.
```

```
Fixpoint t2l t σ (cp: alts) : alts :=
match t with
| OK      ⇒ [(σ, ∅)]
| KO      ⇒ ∅
| TA a    ⇒ [(σ, [(a, ∅)])]
...
```

Figure 9 Valid tree and Tree to stack

mapped to the empty list of alternatives. Finally, atoms are mapped to a singleton list containing one alternative whose only goal is that atom, with an empty cut-to list. At this stage, the cut-to list cannot point to *cp*: atoms are not right-uncle subtrees, whereas *cp* represents alternatives that will actually be discarded if the atom turns out to be a cut. The correct cut-to list is therefore determined later, once the trees corresponding to sibling subtrees have been inspected.

The translation of $A \vee B_\sigma$ creates two list of alternatives, one for the A and the other for B. The alternatives of B are valid choice points for A and should be passed to the translation of A. The two lists of alternatives can then be concatenated. Moreover, every atom occurring one level below *cp* (i.e. in A or B) should add *cp* as its cut-to alternatives.

The translation of $A \wedge_{B_0} B$ starts with the translation of A. If the translation of A is an empty list we return the empty list of alternatives without even touching B nor B_0, since an empty translation for A indicates failure and this in turn implies the failure of the entire conjunction. When the translation of A is a list $(\sigma_0, g) :: a$, the B node represents the continuation of the first alternative g, whereas B_0 is the continuation for each alternative in a.

5.2.1 Example of `t2l` execution

The translations of the trees in Figure 5 are shown in Figure 8. The letters in the figures relate each tree to the corresponding stack. For example, 5b is translated into 8b. We also note that 5d.1 and 5d.2 are both translated into 8d, showing that *t2l* is not injective: different trees can map to the same stack. Consequently, there are transitions in `runT` that are not visible in `runS`. As an example of a call to `t2l`, we take the tree and the stack in Figure 10, which are respectively taken from Figures 5c and 8c. The red arrows point from the head of an alternative in the tree to the head of the corresponding cell in the stack. We focus on the deepest `Or` node in the tree. This subtree is a disjunction with four children. The first is ignored, since it is □. The success node below σ_3 has `f B C` and the cut operator as continuation, corresponding to the first cell in the stack. We

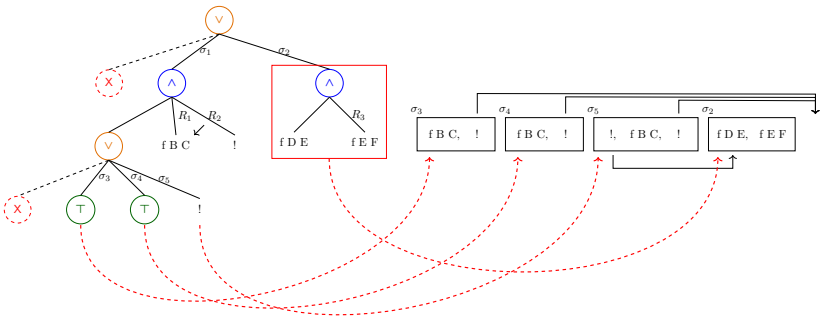


Figure 10 Example of `t2l` execution

note that the cut operator points outside the stack, since the scope of this cut eliminates its right uncle. The success node below σ_4 has R_1 as continuation, where R_1 is the list of goals `f B C, !`; this is translated into the second cell of the stack. Finally, the cut operator under σ_5 also has R_1 as continuation. It corresponds to the third alternative. We can see that the first cut points to the translation of the subtree under σ_2, since it is one `Or`-layer above its right uncles.

5.3 Equivalence proof

The tree-based semantics exposes more information than needed, hence we hide it behind existentials.

▶ **Definition 12** (runT' p v σ t r). $\exists\ b\ v',$ *runT p v σ t r b v'*.

From Figures 5 and 8, we observe that, upon backtracking, `runT` may perform more steps than `runS` before returning a solution or a failure. These silent transitions must be skipped when proving the equivalence between the two semantics. Therefore, we prove the following lemma.

▶ **Lemma 13** (pruneF_t2l). $\forall\ t\ t'\ b,$ *valid_tree t* $\rightarrow$ *failed t* $\rightarrow$ *prune b t* $= \cdot t' \rightarrow$
$\forall\ l\ \sigma,$ *t2l t σ l* = *t2l t' σ l*.

Proof. By induction on the tree t. ◀

The first direction of the equivalence states that the execution of a valid tree is matched by the execution of the stack obtained by translating the tree. Moreover, the unexplored alternatives returned as a tree correspond to those returned as a stack.

▶ **Theorem 14** (tree_to_elpi). $\forall$ *p t σ r, let v := vars_tree t ` | ` vars_sigma σ in*

```
valid_tree t →
  runT' p v σ t r →
    let a := t2l t σ ∅  in
    let r' :=  if r is ·(σ', t) then  ·(σ', t2l (odflt KO t) σ ∅ )
               else □ in
    runS p v a r'.
```

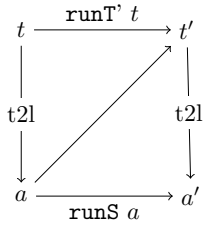
Proof. We proceed by induction on the execution of `runT`'. There are four cases to consider. The first case arises from *StopT*; it deals with successful trees. A successful tree must start with an empty list, and therefore we conclude using *StopS*. The case for *StepT* requires treating two subcases: `step` evaluates either a cut or a call. Accordingly, we apply *CutS* or *CallS*, followed by the induction hypothesis. The case for *BackT* is silent: it represents the non-injective behavior of `t2l`; however, thanks to Lemma 13 and the induction hypothesis, the conclusion is proved. The last case arises from the derivation *FailT*. It is easily proved using *FailS* and the fact that if `prune` returns $\square$ when trying to erase failure in t, then `t2l` applied to t returns the empty list. ◀

The converse direction is stated following [2]: instead of using a function to translate a stack into a tree it states that any tree that translates to the given stack has an equivalent execution, i.e. gives the same solution and returns a unexplored tree that translates to the unexplored stack.

▶ **Theorem 15** (elpi_to_tree). $\forall$ *p v a r, runS p v a r* $\rightarrow$

```
∀ σ t, valid_tree t → t2l t σ ∅  = a →
  if r is ·(σ', a') then
    ∃ t', runT' p v σ t ·(σ', t') ∧ t2l (odflt KO t') σ ∅  = a'
  else runT' p v σ t □.
```

Proof. The proof is based on the idea explained in [2, Section 3.5.1] and depicted in the figure on the right: we have a tree t whose translation is a, we proceed by induction on the execution of `runS`. This gives us not only the output a' but that hypothesis that it exists a tree t' such that its translation to a stack is a'. ◀



Stating the converse direction in a symmetric way would require a function `l2t` transforming a stack a into a tree t. Writing this function is far from trivial, since the structure of the tree is lost by `save_as` and `runT` that intuitively keep the state as a big disjunction of conjunctions by distributing conjunctions over disjunctions. Moreover, one would need to define a notion of valid stack that is equally intricate.

The constructive nature of the logic of Rocq tells us that the proof above provides an effective function translating stacks into trees. However, that function also uses an execution trace of the stack-based semantics, from which it recovers the missing tree structure.

6 Case study: determinacy analysis

An interesting application of the tree semantics is determinacy checking, introduced in version 3 of the Elpi language [12]. The checker takes as input the signatures of the predicates declared by the user and verifies that the rules of a deterministic predicate generate at most one solution; we call this particular kind of predicate a function. Thanks to this static verification, the user can better control unexpected backtracking.

A predicate behaves deterministically if two conditions are satisfied: *mutual exclusion* guarantees that at most one rule can be successfully applied to any given query, whereas *rule checking* ensures that each rule does not create choice points, for example by calling non-deterministic predicates.

Determinacy checking is strongly related to the modes of predicates: we differentiate between input and output arguments. A deterministic call relates to its inputs only one output. In the literature, we find several works on determinacy checking, namely [22, 17, 7]. These works need a mode checker that statically ensures that, in a query with ground inputs, each recursive call is performed with ground inputs.

In our mechanization, and in particular in the Elpi language, we adopt a special unification routine for input arguments, called `matching` [1, 28]. Similarly to `unify`, `matching` takes the query term t_1 and the pattern t_2 and returns an optional substitution σ' that does not assign any variable in the query.

The function `matching` has the following property:

▶ **Axiom 1** (matching_disj). $\forall\ \sigma\ \sigma'\ t_1\ t_2,\ \mathit{vars}\ t_1\ \#\ \mathit{domf}\ \sigma \to \mathit{vars}\ t_1\ \#\ \mathit{vars}\ t_2 \to \mathit{matching}\ t_1\ t_2\ \sigma = \cdot\sigma' \to \exists\ e,\ \mathit{domf}\ \sigma' = \mathit{domf}\ \sigma \cup e\ \wedge\ e \subseteq \mathit{vars}\ t_2.$

If the variables (`vars`) of two terms are disjoint (#) and the variables in the support of σ are disjoint from the variables in t_1, then, if matching succeeds, only the variables in t_2 are assigned; that is, $t_1 = \sigma'\ t_2$, and the variables in t_1 are not assigned in σ'.

The `backchain` function uses `matching` or `unify` on the arguments q depending if the argument is in *input* or *output* mode. We assume two important properties of our unification routines.

▶ **Axiom 2** (unif_trans). $\forall\ t_1\ t_2\ t_3\ \sigma,\ \mathit{unify}\ t_1\ t_2\ \sigma \to \mathit{unify}\ t_2\ t_3\ \sigma \to \mathit{unify}\ t_1\ t_3\ \sigma.$

▶ **Axiom 3** (match_unif). $\forall\ t_1\ t_2\ \sigma,\ \mathit{matching}\ t_1\ t_2\ \sigma \to \mathit{unify}\ t_1\ t_2\ \sigma.$

We also assume that refreshing variables does not affect unification. In particular, a refreshing renaming f for a term t is a function from variables to variables that: (i) f is defined on all the variables in t, i.e. all variables get renamed; (ii) f is injective, i.e. does not confuse variables; and (iii) f has non-overlapping domain and codomain, i.e. it maps the variables of the term to a set of completely fresh variables.

Below, we define `refresh_for` to test the validity of a refreshing renaming. Axiom 4 states the property of term unifiability with respect to term renaming. That is, if the renamings of terms t_1 and t_2 are unifiable under two respective valid refreshing renamings, and they have disjoint variables, then for any pair of renamings z and x that are valid renamings for t_1 and t_2, respectively, and are disjoint from each other, unification also succeeds. In other words, this theorem accounts for α-equivalence of terms and the unification of terms with disjoint variables.

▶ **Definition 16** (refresh_for `x` t). $(\mathit{vars}\ t \subseteq \mathit{domf}\ x) \wedge \mathit{injective}\ x \wedge (\mathit{domf}\ x\ \#\ \mathit{codomf}\ x).$

▶ **Axiom 4** (unif_ren). $\forall\ x\ y\ z\ w\ t_1\ t_2,$

$$\begin{aligned}&\mathit{refresh\_for}\ w\ t_1 \to \mathit{refresh\_for}\ y\ t_2 \to \mathit{refresh\_for}\ z\ t_1 \to \mathit{refresh\_for}\ x\ t_2 \to\\ &\mathit{codomf}\ w\ \#\ \mathit{vars}\ (\mathtt{rename}\ y\ t_2) \to \mathit{codomf}\ z\ \#\ \mathit{vars}\ (\mathtt{rename}\ x\ t_2) \to\\ &\mathit{unify}\ (\mathtt{rename}\ w\ t_1)\ (\mathtt{rename}\ y\ t_2)\ \varepsilon \to \mathit{unify}\ (\mathtt{rename}\ z\ t_1)\ (\mathtt{rename}\ x\ t_2)\ \varepsilon.\end{aligned}$$

In Figure 1, the rules of the program are equipped with three signatures, one per predicate. Besides the arity of each predicate, a signature specifies which arguments are inputs and which are outputs, their types, and whether the predicate is deterministic. They indicate that `f` and `g` are expected to be functions, as specified by the `func` keyword, whereas `r` is a relation, as indicated by the `pred` keyword. The `->` symbol separates inputs from outputs; therefore, all three predicates are expected to take an `int` as input and return an `int` as output.

The signatures of the program are stored in the `sig` field of the program and are encoded as described in [12]; that is, similarly to the code in Figure 1, we use arrows for term application, data types to encode base types such as *int*, and two dedicated symbols to distinguish deterministic from non-deterministic predicates.

A program execution behaves deterministically if, given a program, a substitution, and an input tree for the `runT` inductive, the execution either fails or, if it succeeds, the resulting tree of alternatives is $\square$.

▶ **Definition 17** (is_det p σ v t). $\forall\ r,\ \mathit{runT'}\ p\ v\ \sigma\ t\ r \to r = \square \vee \exists\ \sigma,\ r = \cdot(\sigma,\ \square).$

The theorem we want to prove is the following one:

▶ **Theorem 18** (det_check_call). $\forall\ p\ \sigma\ t\ v,$

$$\mathit{check\_}\mathbb{P}\ p \to \mathit{tm\_is\_det}\ p.(\mathit{sig})\ t \to \mathit{is\_det}\ p\ \sigma\ v\ (\mathit{Todo}\ (\mathit{call}\ t)).$$

Here, `check_`$\mathbb{P}$ denotes the function that checks the program with respect to mutual exclusion and rule checking, and `tm_is_det` denotes the function that tests whether the head of the term refers to a rigid constant with a deterministic type.

The proof of this lemma proceeds by induction on the program execution. However, without further work, the induction hypothesis is too weak since it talks about `Todo` trees (i.e. atoms), whereas we need it on any tree.

To address this issue, we introduce the notion of deterministic trees (`det_tree`), show that a tree satisfying this condition enjoys the desired properties, and then prove the following more general theorem:

▶ **Lemma 19** (det_check_tree). $\forall\ \sigma\ v\ p\ t,\ \mathit{check\_}\mathbb{P}\ p \to \mathit{det\_tree}\ p.(\mathit{sig})\ t \to \mathit{is\_det}\ p\ \sigma\ v\ t.$

Our target theorem `det_check_call` is a corollary of `det_check_tree`.

7 Related work

We studied a Prolog semantics in which input arguments are *matched*, rather than unified, against rule heads. This choice agrees with the Elpi interpreter, which is in turn inspired by B-Prolog's "matching-clauses" [1, 28]. The two semantics we presented (and proved equivalent) also apply to standard Prolog if one forces all predicate signatures to have only output arguments.

For determinacy analysis, matching input arguments affects the proofs only as follows: Axiom 1 does require the query q to be ground. Adapting our results to standard Prolog therefore requires the insertion of two additional hypotheses in Theorem 18. First, one has to require that t has ground inputs. Second, that the program p is well-moded. Well-moded predicates produce ground outputs when given ground inputs; hence, starting from an initial query with ground inputs, well-moded programs behave exactly as if input arguments were matched. Therefore, the proofs in Section 6 still apply.

The only mechanization of Prolog's semantics we could find is the one by Pusch in Isabelle/HOL [21] that is based on the model of Debray and Mishra [6, Sec. 4.3]. Their work starts from that semantics and refines it by introducing some optimizations usually found in the

Warren abstract machine [27, 1]. They do not use the semantics to prove properties on the execution of programs as we do in our use case, hence they do not face the difficulty described in Section 4 that pushed us to develop a more explicit semantics (with respect to cut). In a sense, our work is complementary. By combining both works one can show the tree-based semantics is equivalent to an optimized stack-based one.

Another relevant work is by King [16], but we could not find their Rocq source code. Their semantics is denotational and cannot easily cover all the features that [12] covers, but it would have been sufficient for this first step.

The proof technique we use to relate the two semantics is inspired by [2], and we thank Y. Bertot for insightful discussions on the topic. In [2], Bertot mechanizes the correctness of a compiler. He relates the semantics of an imperative language to that of its compiled assembly code and proves their equivalence. His approach uses the compiler as the translation function from one language to the other, but does not introduce a decompiler, just as we do not define a function from stacks to trees. To show that the assembly code behaves like the imperative language, he proceeds by induction on the execution of the assembly program. We adopt the same strategy in the proof of Theorem 15.

8 Conclusion

This paper describes two operational semantics for Prolog with cut. The first semantics is based on And–Or trees: it is well suited to graphically illustrating the scope of cut, and to proving lemmas about the impact of its evaluation when reasoning about disjunctions. The second semantics is based on a stack of alternatives: it is computationally more efficient, but it loses the structural expressiveness of the tree-based semantics. Thanks to a dedicated function, we translate trees into stacks and prove the two semantics equivalent. This stack-based semantics corresponds to the first-order fragment of Elpi. Using the tree-based semantics, we mechanize the first-order part of the determinacy checker that has been available in Elpi since version 3.0, and we prove that a call to a deterministic predicate returns at most one solution.

The formal development consists of approximately 2,500 lines of specifications and about 5,000 lines of *ssreflect* proofs. Roughly 30% of the code is dedicated to the tree semantics (in particular, the proofs in Section 3.4), 15% to the stack semantics, and 35% to the equivalence proof. The remaining 20% is devoted to the determinacy checker. The mechanization took us approximately 8 months.

To fully model the Elpi language and mechanize [12], we would need to account for higher-order variables, i.e. variables representing predicates. This extension is ongoing and largely orthogonal to the present paper, since cut and higher-order features do not interact in subtle ways.

References

1 Hassan Aït-Kaci. *Warren's Abstract Machine: A Tutorial Reconstruction*. The MIT Press, 08 1991. doi:10.7551/mitpress/7160.001.0001.

2 Yves Bertot. A certified compiler for an imperative language. Technical Report RR-3488, INRIA, September 1998. URL: https://inria.hal.science/inria-00073199v1.

3 Valentin Blot, Denis Cousineau, Enzo Crance, Louise Dubois de Prisque, Chantal Keller, Assia Mahboubi, and Pierre Vial. Compositional pre-processing for automated reasoning in dependent type theory. In Robbert Krebbers, Dmitriy Traytel, Brigitte Pientka, and Steve Zdancewic, editors, *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2023, Boston, MA, USA, January 16-17, 2023*, pages 63–77. ACM, 2023. doi:10.1145/3573105.3575676.

4 Cyril Cohen, Enzo Crance, and Assia Mahboubi. Trocq: Proof transfer for free, with or without univalence. In Stephanie Weirich, editor, *Programming Languages and Systems*, pages 239–268, Cham, 2024. Springer Nature Switzerland.

5 Cyril Cohen, Kazuhiko Sakaguchi, and Enrico Tassi. Hierarchy Builder: Algebraic hierarchies Made Easy in Coq with Elpi. In *Proceedings of FSCD*, volume 167 of *LIPIcs*, pages 34:1–34:21, 2020. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSCD.2020.34, doi:10.4230/LIPIcs.FSCD.2020.34.

6 Saumya K. Debray and Prateek Mishra. Denotational and operational semantics for prolog. *J. Log. Program.*, 5(1):61–91, March 1988. doi:Debray1988.

7 Saumya K. Debray and David S. Warren. Functional computations in logic programs. volume 11, page 451–481. Association for Computing Machinery, July 1989. doi:10.1145/65979.65984.

8 Gilles Dowek. *Higher-order unification and matching*, page 1009–1062. Elsevier Science Publishers B. V., NLD, 2001.

9 Cvetan Dunchev, Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. ELPI: fast, embeddable, λProlog interpreter. In *Proceedings of LPAR*, volume 9450 of *LNCS*, pages 460–468. Springer, 2015. URL: https://inria.hal.science/hal-01176856v1, doi:10.1007/978-3-662-48899-7_32.

10 Davide Fissore and Enrico Tassi. A new Type-Class solver for Coq in Elpi. In *The Coq Workshop*, July 2023. URL: https://inria.hal.science/hal-04467855.

11 Davide Fissore and Enrico Tassi. Higher-order unification for free!: Reusing the meta-language unification for the object language. In *Proceedings of PPDP*, pages 1–13. ACM, 2024. doi:10.1145/3678232.3678233.

12 Davide Fissore and Enrico Tassi. Determinacy checking for elpi: an higher-order logic programming language with cut. In Nada Amin and Joaquín Arias, editors, *Practical Aspects of Declarative Languages*, pages 77–95, Cham, 2026. Springer Nature Switzerland.

13 Benjamin Grégoire, Jean-Christophe Léchenet, and Enrico Tassi. Practical and sound equality tests, automatically. In *Proceedings of CPP*, page 167–181. Association for Computing Machinery, 2023. doi:10.1145/3573105.3575683.

14 Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. Implementing type theory in higher order constraint logic programming. In *Mathematical Structures in Computer Science*, volume 29, pages 1125–1150. Cambridge University Press, 2019. doi:10.1017/S0960129518000427.

15 Robbert Krebbers, Luko van der Maas, and Enrico Tassi. Inductive Predicates via Least Fixpoints in Higher-Order Separation Logic. In Yannick Forster and Chantal Keller, editors, *16th International Conference on Interactive Theorem Proving (ITP 2025)*, volume 352 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 27:1–27:21, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2025.27, doi:10.4230/LIPIcs.ITP.2025.27.

16 Jael Kriener and Andy King. Redalert: Determinacy inference for prolog. volume 11, pages 537–553. Cambridge University Press, 2011.

17 Lunjin Lu and Andy King. Determinacy inference for logic programs. volume 3444, pages 108–123, 04 2005. doi:10.1007/978-3-540-31987-0_9.

18 math-comp. finmap — finite maps for mathcomp, 2026. URL: https://github.com/math-comp/finmap.

19 Dale Miller. A logic programming language with lambda-abstraction, function variables, and simple unification. In *Extensions of Logic Programming*, pages 253–281. Springer, 1991.

20 Dale Miller and Gopalan Nadathur. *Programming with Higher-Order Logic*. Cambridge University Press, 2012.

21 Cornelia Pusch. Verification of compiler correctness for the wam. In Gerhard Goos, Juris Hartmanis, Jan van Leeuwen, Joakim von Wright, Jim Grundy, and John Harrison, editors, *Theorem Proving in Higher Order Logics*, pages 347–361, Berlin, Heidelberg, 1996. Springer Berlin Heidelberg.

22 Zoltan Somogyi, Fergus Henderson, and Thomas Conway. The execution algorithm of mercury, an efficient purely declarative logic programming language. volume 29, pages 17–64, 1996. High-Performance Implementations of Logic Programming Systems. URL: https://www.sciencedirect.com/science/inproceedings/pii/S0743106696000684, doi:10.1016/S0743-1066(96)00068-4.

23 SWI-Prolog Documentation. Predicate !/0 — cut (built-in predicate), 2026. Accessed via SWI-Prolog PlDoc reference. URL: https://www.swi-prolog.org/pldoc/doc_for?object=!/0.

24 Enrico Tassi. Elpi: an extension language for Coq (Metaprogramming Coq in the Elpi λProlog dialect). In *The Fourth International Workshop on Coq for Programming Languages*, January 2018. URL: https://inria.hal.science/hal-01637063.

25 Enrico Tassi. Deriving proved equality tests in Coq-Elpi. In *Proceedings of ITP*, volume 141 of *LIPIcs*, pages 29:1–29:18, September 2019. URL: https://inria.hal.science/hal-01897468, doi:10.4230/LIPIcs.CVIT.2016.23.

26 Luko van der Maas. Extending the Iris Proof Mode with inductive predicates using Elpi. Master's thesis, Radboud University Nijmegen, 2024. doi:10.5281/zenodo.12568604.

27 David H.D. Warren. An Abstract Prolog Instruction Set. Technical Report Technical Note 309, SRI International, Artificial Intelligence Center, Computer Science and Technology Division, Menlo Park, CA, USA, October 1983. URL: https://www.sri.com/wp-content/uploads/2021/12/641.pdf.

28 Neng-fa Zhou. The language features and architecture of b-prolog. *Theory Pract. Log. Program.*, 12(1–2):189–218, January 2012. doi:10.1017/S1471068411000445.